Part 1: The FHE Dream and the Integer Approach

Speaker Notes & Mathematical Primer

Nirav Bhattad

March 27, 2026

§1. Motivation: The Cloud Computing Conundrum

How to start speaking: "Welcome, everyone. Today we are going to dive into one of the most elegant and conceptually striking papers in modern cryptography: *Fully Homomorphic Encryption over the Integers* by van Dijk, Gentry, Halevi, and Vaikuntanathan (2010).
Before we look at the math, let's establish the fundamental problem we are trying to solve. In modern computing, we have perfectly good solutions for protecting data *in transit* (like TLS/SSL) and data *at rest* (like AES). But what happens when we want to compute on that data? Traditionally, to perform a computation, you have to decrypt the data first. This creates a massive vulnerability: if you want to outsource heavy computations to an untrusted cloud server, you have to give them your secret key or your raw data."

The goal of Fully Homomorphic Encryption (FHE) is to allow an untrusted party (like a cloud server) to take encrypted inputs, perform arbitrary, unbounded computations on them, and return an encrypted result. When the data owner decrypts this result, it matches exactly what they would have gotten had they performed the computation on the plaintext themselves.

The Analogy (The Caesar Cipher): "The authors open the paper with a provocative question: *What is the simplest encryption scheme for which one can hope to achieve security?*
Think of the Caesar cipher. It is incredibly simple, and ironically, it is somewhat homomorphic! If you shift 'A' by 3 and 'B' by 3, you can easily add those shifts together. But it is entirely insecure. On the other hand, RSA and ElGamal are secure, but they rely on modular exponentiation—which is computationally complex—and they only support one type of operation (multiplication). You can't evaluate an entire boolean circuit using just multiplication. We need both addition and multiplication (an algebraically complete set of gates) to compute any arbitrary function."

§2. Formalizing the FHE Dream

To understand the magnitude of the problem, we must rigorously define what a homomorphic encryption scheme entails. A homomorphic public key encryption scheme, denoted $\mathcal{E}$, consists of four probabilistic polynomial-time (PPT) algorithms:

$$\mathcal{E} = (\mathsf{KeyGen}, \mathsf{Encrypt}, \mathsf{Decrypt}, \mathsf{Evaluate}) \tag{2.1}$$

The standard first three algorithms operate as usual. The magic lies in Evaluate. The algorithm Evaluate takes as input a public key pk, a boolean circuit C, and a tuple of ciphertexts $\vec{c} = \langle c_1, \ldots, c_t \rangle$ (corresponding to the t input bits of C), and outputs a new ciphertext c_{out}.

Definition 2.1: Correct Homomorphic Decryption

The scheme $\mathcal{E}$ is correct for a given t-input circuit C if, for any key-pair $(sk, pk) \leftarrow \mathsf{KeyGen}(\lambda)$, any plaintexts $m_1, \ldots, m_t \in \{0, 1\}$, and any ciphertexts $c_i \leftarrow \mathsf{Encrypt}(pk, m_i)$, the following holds with overwhelming probability over the randomness of the algorithms:

$$\mathsf{Decrypt}(sk, \mathsf{Evaluate}(pk, C, \vec{c})) = C(m_1, \ldots, m_t) \tag{2.2}$$

Definition 2.2: Fully Homomorphic Encryption

The scheme $\mathcal{E}$ is homomorphic for a class $\mathcal{C}$ of circuits if it is correct for all circuits $C \in \mathcal{C}$. The scheme $\mathcal{E}$ is **fully homomorphic** if it is correct for all boolean circuits (i.e., circuits of arbitrary depth and size).

Addressing the Trivial Solution: "Now, you might be thinking: I can build a scheme that perfectly satisfies this definition right now. How? My Evaluate algorithm will just take the circuit C and concatenate it to the ciphertexts $c_1, \ldots, c_t$. It does zero actual computation. Then, my Decrypt algorithm will decrypt the inputs, read the attached circuit C, and run the computation itself.
Does this satisfy the definition? Yes. Is it useful? Absolutely not. It entirely defeats the purpose of outsourcing computation, because the data owner does all the work! Therefore, FHE has a strict constraint: **Compactness**."

Definition 2.3: Compactness

A homomorphic scheme $\mathcal{E}$ is *compact* if there exists a fixed polynomial bound $b(\lambda)$ such that for any generated key-pair, any circuit C, and any sequence of ciphertexts $\vec{c}$, the size of the ciphertext output by $\mathsf{Evaluate}(pk, C, \vec{c})$ is strictly bounded by $b(\lambda)$ bits, **independently of the size or depth of** C.

§3. The Paradigm Shift: From Ideal Lattices to Integers

The Hook for the Audience: "For 30 years after Rivest, Adleman, and Dertouzos first proposed the idea of privacy homomorphisms in 1978, nobody knew if FHE was mathematically possible.
Then, in 2009, Craig Gentry shocked the world. He proved FHE was possible. But Gentry's blueprint was notoriously difficult to understand. It relied on complex algebraic geometry, specifically working with ideal lattices over polynomial rings. To even begin understanding his paper, you needed a strong background in algebraic number theory.
This brings us to our paper today. Published just a year later in 2010, the authors asked a brilliant question: *Did Gentry need all that heavy machinery?* The answer is no."

The landmark contribution of the DGHV paper is not just achieving FHE, but doing so with radical conceptual simplicity. Instead of working with ideal lattices over a polynomial ring, the DGHV scheme constructs a "somewhat homomorphic" encryption scheme that merely uses **addition and multiplication over the integers**.
By stripping away the complex algebraic structures, the authors proved that the "blueprint" Gentry established—building a somewhat homomorphic scheme, squashing the decryption circuit, and then bootstrapping it—can be instantiated using elementary modular arithmetic.

Setting up Part 2: "The beauty of this paper is that we can understand state-of-the-art cryptography using math you learned in high school and your first discrete math course. The security of this scheme will not rely on finding short vectors in a lattice. Instead, it relies on a much more fundamental, intuitive number-theoretic puzzle: the **Approximate-GCD problem**.
If I give you a set of integers that are exact multiples of a secret prime p, finding p is trivial using the Euclidean algorithm. But what if I add just a tiny bit of random noise to each multiple? Suddenly, the problem becomes exponentially hard. That simple idea is the bedrock of the entire scheme we are about to explore."